

# **OMNIBRAIN**

**Agentic Multi-Modal RAG Orchestrator**

## **PROJECT REPORT**

Domain: Large Language Models (LLMs) & Agentic AI

**Submitted By**

**BURRI SHIRISHA**

# 1. Declaration

This project report presents OmniBrain, an agentic multi-modal Retrieval-Augmented Generation (RAG) prototype designed for analyzing complex financial documents. The system combines document retrieval, structured SQL querying, visual reasoning, and grounded language-model synthesis under an agentic orchestration layer.

## 2. Abstract

Traditional RAG systems primarily retrieve text and can struggle when important information is contained in financial tables, charts, images, or when a question requires reasoning across multiple data sources. OmniBrain addresses this challenge through an agentic architecture based on LangGraph. A supervisor routes requests to specialized Search, SQL, and Vision capabilities. Retrieved evidence is combined and passed to a grounded language model that produces a cited research response. The prototype is intended as a foundation for financial research and can be extended with Qdrant, CLIP, Langfuse, NeMo Guardrails, and production market databases.

## 3. Problem Statement

Standard RAG pipelines can fail to capture information represented visually and may provide incomplete answers when a query requires multiple reasoning and retrieval steps. Financial reports frequently contain hundreds of pages with tables, charts, figures, and numerical information. A system is therefore required that can select the appropriate retrieval mechanism, combine evidence from different sources, and generate an answer grounded in retrieved context.

## 4. Objectives

- Build an agentic supervisor using LangGraph.
- Extract and index text from long financial PDFs.
- Support semantic retrieval using embeddings and FAISS.
- Extract images and provide a Vision-Language Model interface.
- Query structured historical stock data using SQL.
- Combine evidence from specialized agents.
- Generate grounded answers with citation markers.
- Provide a modular foundation for evaluation and guardrails.

## 5. Proposed System

OmniBrain receives a natural-language question from a quantitative analyst. The LangGraph supervisor determines which capabilities are required. The Search Agent retrieves relevant document passages, the SQL Agent retrieves structured historical market information, and the Vision Agent analyzes relevant charts or images. The evidence is then supplied to a synthesis model that produces a grounded answer.

## 6. System Architecture

User Query → LangGraph Supervisor → Specialized Agents → Evidence Aggregation → Grounded LLM → Cited Investment Memo

### Specialized Agents

- 1. Search Agent — semantic text retrieval from indexed PDF content.
- 2. SQL Agent — structured querying of historical stock data.
- 3. Vision Agent — analysis of charts and embedded images using a VLM.

## 7. Technology Stack

| Component                       | Technology                              |
|---------------------------------|-----------------------------------------|
| Agent Orchestration             | LangGraph                               |
| Language Model                  | OpenAI-compatible LLM / GPT-4o family   |
| Vision Model                    | GPT-4o-compatible Vision-Language Model |
| Text Embeddings                 | Sentence Transformers                   |
| Vector Search                   | FAISS                                   |
| PDF Processing                  | PyMuPDF                                 |
| Structured Data                 | SQLite / SQL                            |
| API Layer                       | FastAPI                                 |
| Programming Language            | Python                                  |
| Potential Production Vector DB  | Qdrant                                  |
| Potential Multimodal Embeddings | CLIP                                    |
| Observability                   | Langfuse                                |
| Guardrails                      | NeMo Guardrails / custom validation     |

## 8. Module Description

### 8.1 Agentic Orchestrator

The LangGraph supervisor manages workflow state and routing. It examines the user query and decides whether semantic retrieval and structured market-data retrieval are required. The graph-based design makes it straightforward to add additional agents in the future.

### 8.2 Document Ingestion and Retrieval

The ingestion module uses PyMuPDF to extract text from PDF pages and identify embedded images. Text chunks are transformed into embeddings using a Sentence Transformer model and indexed using FAISS for similarity search.

### 8.3 Vision Agent

The Vision Agent accepts an extracted image and a question. When an OpenAI-compatible vision model is configured, the image is encoded and sent to the VLM for interpretation. This enables analysis of charts, graphs and other visual information.

### 8.4 SQL Agent

The SQL Agent demonstrates access to historical stock information using SQLite. The prototype can be replaced with a production market-data database. Production deployment should enforce read-only access, schema validation and parameterized queries.

### 8.5 Grounded Synthesis

The synthesis stage receives retrieved evidence and instructs the language model to answer only from the available context. Citation markers such as [1] and [2] are used to associate statements with retrieved evidence.

### 8.6 Evaluation and Guardrails

The project contains a basic guardrail module for empty responses and simple unsafe prompt patterns. A production version can integrate Langfuse for tracing and evaluation and NeMo Guardrails for policy enforcement and controlled conversations.

## 9. Project Workflow

1. User submits a financial research question.
2. LangGraph supervisor analyzes the question.
3. Relevant document passages are retrieved through the Search Agent.
4. If market information is required, the SQL Agent queries structured data.
5. If visual information is required, the Vision Agent analyzes selected images.
6. The evidence is aggregated into a common context.
7. The grounded LLM generates the final response.
8. Citation markers are included to improve traceability.
9. Guardrails and evaluation can be applied before presenting the answer.

## 10. Implementation Details

The source package is organized into independent modules: **ingest.py** handles PDF processing and indexing, **agents.py** contains Search, SQL, Vision and synthesis logic, **graph.py** defines the LangGraph workflow, **app.py** exposes the FastAPI endpoint, and **guardrails.py** contains basic validation utilities.

## 11. Installation

Create a Python virtual environment, install the packages from requirements.txt, configure the API key in .env, ingest a financial PDF, and start the FastAPI server using Uvicorn. The API documentation is then available through the FastAPI /docs page.

## 12. Expected Output

For a question such as 'What was the revenue trend and how does it compare with historical stock performance?', OmniBrain can retrieve relevant financial-report passages and structured stock records, combine them, and return an evidence-grounded answer with citation markers.

## 13. Advantages

- Supports multiple retrieval modalities.
- Agent routing avoids using every tool for every question.
- Modular architecture allows additional agents to be added.
- Semantic retrieval improves access to relevant PDF content.
- SQL access supports structured numerical data.
- Vision capability can interpret charts and images.
- Grounded synthesis reduces unsupported statements.
- FastAPI provides a simple interface for integration.

## 14. Limitations

The supplied implementation is an academic prototype. It does not guarantee zero hallucinations or production-level financial accuracy. Qdrant, CLIP, Langfuse and NeMo Guardrails are identified as production extensions rather than fully configured services in the starter implementation. Real financial data must be loaded into the SQL database before meaningful market-data results can be produced.

## 15. Future Enhancements

- Replace FAISS with Qdrant for scalable multimodal vector storage.
- Add CLIP or another multimodal embedding model.
- Implement automatic table extraction and table-aware retrieval.
- Add a dedicated Text-to-SQL generation and validation agent.
- Integrate Langfuse tracing, metrics and evaluation datasets.
- Integrate NeMo Guardrails for stronger policy enforcement.
- Add reranking and hybrid keyword/vector retrieval.
- Add authentication, caching and production deployment.
- Add financial-data connectors and scheduled data updates.

## 16. Conclusion

OmniBrain demonstrates an agentic approach to multimodal financial RAG. By combining LangGraph orchestration with semantic retrieval, SQL access, vision-language reasoning and grounded synthesis, the system provides a practical foundation for analyzing complex financial reports. The modular architecture also provides clear paths toward production features such as Qdrant, CLIP, Langfuse and NeMo Guardrails.

## 17. References / Technologies

LangGraph; FAISS; Sentence Transformers; PyMuPDF; FastAPI; SQLite; OpenAI-compatible LLM/VLM APIs; Qdrant; CLIP; Langfuse; NeMo Guardrails.

**Submitted By: Burri Shirisha**